



7. Python: Loops

"Do more with less code."

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 6. Python: Built-in Methods](#)

Contents:

[1. What are Loops?](#)

[For Loop](#)

[While Loop](#)

[2. Nested Loops](#)

[3. Loops with Conditional Statements](#)

[Summary](#)

[References](#)

Programs often need to repeat the same instructions multiple times. Instead of writing the same code again and again, Python provides **loops** to automate repetition efficiently. Loops are commonly used for traversing sequences, drawing patterns, processing data, and performing repetitive calculations.

Python mainly provides two types of loops: **For Loop and While Loop**

1. What are Loops?

Loops are structures that repeatedly execute a block of code until a condition is satisfied or until all elements in a sequence are processed.

For Loop

A **For Loop** is used to iterate through sequences such as lists, tuples, strings, dictionaries, or ranges of numbers.

Basic Syntax:

```
for variable in sequence:  
    # code
```

- Using `range()`: generates values from `0` to `n - 1`.

```
# Iterates from 0 to 4  
for i in range(5):  
    print(i)
```

0
1
2
3
4

- You can also specify a **starting point, ending point, and step size**.

```
# Iterates from 2 to 10 with step size 2  
for i in range(2, 11, 2):  
    print(i)
```

2
4
6
8
10

- Iterating Through a List

```
s = [1, 2, 3, 4, 5, 6]

for i in range(len(s)):
    print(s[i])
```

```
1
2
3
4
5
6
```

- **Printing on the Same Line:** The `end=` parameter prevents `print()` from moving to a new line.

```
s = [1, 2, 3, 4, 5, 6]

for i in range(len(s)):
    # The end= puts all output in a single row
    print(s[i], end=' ')

print() # Create a space

for j in range(len(s)):
    print(s[j], end=' & ')
```

```
1 2 3 4 5 6
1 & 2 & 3 & 4 & 5 & 6 &
```

- **Naming the Loop Variable:** The loop variable can have any valid name.

```
fruits = ['apple', 'banana', 'orange', 'mango']

for fruit in fruits:
    print(fruit)
```

apple
banana
orange
mango

- Using `enumerate()` : This is a useful function that provides both the index and the value during iteration.

```
fruits = ['apple', 'banana', 'orange', 'mango']

for index, value in enumerate(fruits):
    print(index, value)
```

0 apple
1 banana
2 orange
3 mango

While Loop

A **While Loop** repeatedly executes code while a condition remains `True`.

Basic Syntax:

```
while condition:
    # code
```

Unlike the **For Loop**, a **While Loop** does not have a fixed stopping point, making it more flexible but also more prone to infinite loops if the condition is never changed.

- **Example: Password Checking**

```
# Asking the user for a password until it's correct
password = ""

while password != "myname123":
    password = input("Enter the password: ")
    if password != "myname123":
        print("Wrong password, try again!")

print("Access granted!")
```

Enter the password: myname12
 Wrong password, try again!
 Enter the password: myname1234
 Wrong password, try again!
 Enter the password: myname123
 Access granted!

- **Example: Stop When User Enters 0**

```
# Calculation of all numbers
# Stop when the user enters 0

number = 1

while number != 0:
    number = int(input("Enter a number: "))
    if number != 0:
        print(number)

print("End Loop")
```

Enter a number: 12
 12
 Enter a number: 13
 13
 Enter a number: 0
 End Loop

Useful Loop Methods - Python provides two important loop control statements:

`break` and `continue`

- `break` immediately terminates the loop once a condition is met.

```
i = 10
count = 0

while i > -10:
    if i == 0:
        print("Reaches 0 at iteration:", count)
        break

    count += 1
    i -= 1
```

Reaches 0 at iteration: 10

- Example: Stop When Sum Reaches 1000

```
# Continuously compute 1 + 2 + 3 + ... + n
# Stop when the sum reaches 1000

j = 0

for i in range(1, 101):
    j += i

    if j >= 1000:
        print("Reaches 1000 at number", i)
        break
```

Reaches 1000 at number 45

- `continue` skips the current iteration and moves directly to the next iteration.

```
i = 0

while i < 20:

    if i % 2 != 0:
        i += 1
        continue

    print(i)
    i += 1
```

0
2
4
6
8
10
12
14
16
18

- Example: Print Odd Numbers from 1 to 10

```
for i in range(1, 11):
    if i % 2 == 0:
        continue

    print(i, end=" ")
```

1 3 5 7 9

2. Nested Loops

A **Nested Loop** is a loop placed inside another loop. Nested loops are useful for handling more complex problems such as:

- Traversing matrices or multi-dimensional lists
 - Comparing elements
 - Generating combinations
 - Drawing patterns
 - Repetitive computations inside another repetition process
-
- Basic Nested Loop

```
# Cartesian Product

for i in range(0, 3):
    for j in range(0, 3):
        print(i, j)
```

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

The outer loop controls `i`, while the inner loop fully iterates through `j` for every value of `i`.

Nested Loop with Lists of Lists


```

name = [['John', 23], ['James', 29], ['Chris', 14]]

for i in range(len(name)):
    print(name[i])

    for j in range(len(name[i])):
        print(name[i][j])

```

```

['John', 23]
John
23
['James', 29]
James
29
['Chris', 14]
Chris
14

```

This is commonly used when traversing two-dimensional lists or matrices.

- Multiplication Table

```

# Multiplication table using nested loops
for i in range(2, 10):
    for j in range(2, 6):
        print(f"{i} x {j} = {i * j}", end="\t")

    print()

```

2 × 2 = 4	2 × 3 = 6	2 × 4 = 8	2 × 5 = 10
3 × 2 = 6	3 × 3 = 9	3 × 4 = 12	3 × 5 = 15
4 × 2 = 8	4 × 3 = 12	4 × 4 = 16	4 × 5 = 20
5 × 2 = 10	5 × 3 = 15	5 × 4 = 20	5 × 5 = 25
6 × 2 = 12	6 × 3 = 18	6 × 4 = 24	6 × 5 = 30
7 × 2 = 14	7 × 3 = 21	7 × 4 = 28	7 × 5 = 35
8 × 2 = 16	8 × 3 = 24	8 × 4 = 32	8 × 5 = 40
9 × 2 = 18	9 × 3 = 27	9 × 4 = 36	9 × 5 = 45

The inner loop generates the multiplication expressions, while the outer loop changes the main multiplier.

- **Perfect Number Detection**

A **Perfect Number** is a number whose divisors sum up to the number itself.

Example: Divisors of 6 are 1, 2, 3 $\rightarrow 1 + 2 + 3 = 6$. Therefore, 6 is a perfect number.

```
# Detect perfect numbers
for i in range(1, 10000):
    sum_divisors = 0

    for j in range(1, i):
        if i % j == 0:
            sum_divisors += j

    if sum_divisors == i:
        print(i)
```

```
6
28
496
8128
```

This demonstrates how nested loops are useful for divisor checking and repetitive calculations.

- **Using `zip()` as an Alternative:** The `zip()` function pairs elements from multiple lists based on their positions, allowing iteration over multiple sequences at the same time.

```
fruits = ['apple', 'banana', 'orange', 'mango']
veggies = ['carrot', 'tomato', 'potato', 'cabbage']

for fruit, veg in zip(fruits, veggies):
    print(fruit, veg)
```

```
apple carrot
banana tomato
orange potato
mango cabbage
```

`zip()` can sometimes replace nested loops when iterating through multiple lists together, although nested loops are still necessary for many complex problems.

3. Loops with Conditional Statements

Loops and conditional statements (`if`, `elif`, `else`) are often combined to solve more advanced problems. While the loop handles repetition, conditional statements allow the program to make decisions during each iteration.

This combination is useful for:

- Filtering values
 - Counting specific conditions
 - Searching and comparisons
 - Mathematical computations
 - Decision-making during iteration
-
- Odd and Even Numbers Between 1 and 100

```
# Odd-Even numbers between 1 and 100
```

```
count = 0
```

```
for i in range(1, 101):  
    if i % 2 == 0:  
        print(i, end=" ")  
        count += 1  
  
        if count == 3:  
            print()  
            count = 0
```

```
2 4 6  
8 10 12  
14 16 18  
20 22 24  
26 28 30  
32 34 36  
38 40 42  
44 46 48  
50 52 54  
56 58 60  
62 64 66  
68 70 72  
74 76 78  
80 82 84  
86 88 90  
92 94 96  
98 100
```

This program prints even numbers and formats the output into groups of three numbers per line.

- **Prime Numbers Between 1 and 100**

A **Prime Number** is a number greater than 1 that is divisible only by 1 and itself.

```
# Prime numbers between 1 and 100
```

```
count = 0
```

```
for i in range(2, 101):
```

```
    is_prime = True
```

```
    for j in range(2, i):
```

```
        if i % j == 0:
```

```
            is_prime = False
```

```
            break
```

```
    if is_prime:
```

```
        print(i, end=" ")
```

```
        count += 1
```

```
    if count == 3:
```

```
        print()
```

```
        count = 0
```

```
2 3 5
```

```
7 11 13
```

```
17 19 23
```

```
29 31 37
```

```
41 43 47
```

```
53 59 61
```

```
67 71 73
```

```
79 83 89
```

```
97
```

The inner loop checks whether a number has divisors other than 1 and itself.

- **Counting Numbers Divisible by 3, 6, and 9**

```
# Count numbers divisible by 3, 6, and 9

count1, count2, count3 = 0, 0, 0

for i in range(3, 101):

    if i % 3 == 0:
        count1 += 1

    if i % 6 == 0:
        count2 += 1

    if i % 9 == 0:
        count3 += 1

print("Count of 3:", count1)
print("Count of 6:", count2)
print("Count of 9:", count3)
```

Count of 3: 33
Count of 6: 16
Count of 9: 11

This demonstrates how multiple conditions can be checked during each iteration.

- **Finding the Largest Number in a List**

```
# Find the largest number in a list

sequence = [92, 51, 27, 56, 48, 18, 57, 64, 31, 8]
number = float('-inf')

# Iterate through the list and compare values
for x in sequence:
    if x > number:
        number = x

print("Largest Number:", number)
```

Largest Number: 92

`float('-inf')` represents negative infinity, ensuring the first comparison always succeeds.

- **Sum of Even and Odd Numbers**

```
# Sum of even and odd numbers between 1 and 112

sum_of_even = 0
sum_of_odd = 0

for i in range(1, 112 + 1):
    if i % 2 == 0:
        sum_of_even += i
    else:
        sum_of_odd += i

print("Sum of even:", sum_of_even)
print("Sum of odd:", sum_of_odd)
```

Sum of even: 3192

Sum of odd: 3136

This program separates values into two categories using `if-else`.

- **Greatest Common Divisor (GCD) Using Loops**

The **Greatest Common Divisor (GCD)** is the largest number that divides both numbers exactly.

```
# Greatest Common Divisor using loops

gcd = 0

a = int(input("Enter a number: "))
b = int(input("Enter a number: "))

if a < b:
    for i in range(1, a + 1):
        if (a % i) == 0 and (b % i) == 0:
            gcd = i
    print(f"GCD of {a} and {b} is {gcd}")

elif a > b:
    for i in range(1, b + 1):
        if (a % i) == 0 and (b % i) == 0:
            gcd = i
    print(f"GCD of {a} and {b} is {gcd}")

else:
    print(f"GCD of {a} and {b} is {a}")
```

Enter a number: 12

Enter a number: 36

GCD of 12 and 36 is 12

The loop checks all possible divisors and stores the greatest common one.

Summary

Loops become significantly more powerful when combined with conditional statements. Together, they allow programs to repeatedly process data while

making logical decisions at every step. This combination forms the foundation of many important programming tasks such as searching, filtering, counting, mathematical computation, and algorithm design.

References

<https://www.learnpython.org/en/Loops>

<https://www.geeksforgeeks.org/python/loops-in-python/>

<https://www.codecademy.com/learn/learn-python-3/modules/learn-python3-loops/cheatsheet>

Next Section:

[`</>` 8. Python: Functions](#)